

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

Fall Semester 2026

{{Course Code}} {{Course Title}}

{{Internship - I / Dissertation - I}}

Class ID : {{Class ID}}

Review - 3

Date: {{DD/MM/YYYY}}

CODESHIELD AI: A CONTEXT-AWARE AND DESTINATION-SENSITIVE FRAMEWORK FOR PREVENTING SENSITIVE DATA LEAKAGE IN DEVELOPER AND LLM WORKFLOWS

{{Register Number}} – Bhargav Mahanta

Under the Guidance of

Prof. Nivedita S

Assistant Professor, SCORE

PROJECT SNAPSHOT

PART A – PROJECT OVERVIEW

Particular	Details
Project Title	CodeShield AI: A Context-Aware and Destination-Sensitive Framework for Preventing Sensitive Data Leakage in Developer and LLM Workflows
Area of Application	Software and application security: data-loss prevention for developer tooling, source-code repositories and generative-AI (LLM) coding workflows.
Scope	A local-first prevention layer that scans files, staged Git additions, editor text and structured LLM prompts and responses on the developer machine; classifies each finding by context and sensitivity; applies destination-aware YAML policy and provider-specific LLM rules; and allows, warns, masks, pseudonymizes or blocks content before it crosses a Git or LLM trust boundary. It includes a CLI, loopback API, Git hook, VS Code extension, TypeScript SDK, authenticated LLM proxy and an optional metadata-only management plane. Hosted deployment, streaming proxy support and real-world accuracy claims are outside the current scope.

Alignment with Sustainable Development Goals

SDG No.	SDG Title	Justification
SDG 9	Industry, Innovation and Infrastructure	Makes software-development infrastructure more resilient (target 9.1) by stopping credential and source-code leaks that lead to breaches, and builds local security tooling capability (target 9.5).
SDG 16	Peace, Justice and Strong Institutions	Protects personal data and privacy (target 16.10) by keeping raw values local and pseudonymizing personal data, and supports accountable institutions (target 16.6) through signed policies and metadata-only audit records.

Inputs – Features – Outputs

S. No.	Feature	Input Given	Output
1	Sensitive-data scanning and Git protection	Files, folders, editor text or the added lines of a staged Git diff, with the repository policy (.codeshield.yaml)	Masked findings with category, severity, context and 0–100 risk score; human, JSON or SARIF 2.1.0 output; exit code 0/1/2/3 that lets the pre-commit hook reject a commit
2	Context-aware, destination-sensitive policy	Each finding with its surrounding code context, the destination (local file, private/public Git, log, internal/external LLM, API, cloud storage) and repository visibility	Context label (7 classes), sensitivity (Public/Internal/Confidential/Restricted) and an explainable ALLOW, WARN, MASK, PSEUDONYMIZE or BLOCK decision
3	Local protection and pseudonymization	Text or file, a protection mode (redact, partial mask, type placeholder, pseudonymize) and a vault key from the environment	Protected text; reversible mappings stored only in an AES-GCM encrypted local vault; exact local restoration
4	Provider-aware LLM preflight and proxy	Structured prompt segments or an OpenAI-compatible request, a provider profile and a local attestation	Decision before any network connection; protected outbound request; buffered, inbound-inspected response; metadata-only HMAC audit record

S. No.	Feature	Input Given	Output
5	Metadata-only management plane	Reviewed policy drafts, device enrollment and aggregate audit counts	Immutable Ed25519-signed policy bundles, revocable device tokens and a role-based dashboard
Final Output		A local preflight boundary that stops credentials, Restricted data and unsafe prompts before they reach Git remotes or external LLM providers. Only protected content is sent, while raw values, keys and pseudonym mappings stay on the developer machine.	

Demo Video

Particular	Details
Video URL	{{Video URL}}
Duration	{{mm:ss}}

Technical Stack

Layer	Technology / Tool
Frontend	Next.js 16.2 App Router, React 19.2, Tailwind CSS 4.3 and Recharts 3.10 (management dashboard); VS Code extension in TypeScript
Backend	Python 3.12 engine with Typer CLI, FastAPI and Uvicorn (local service, LLM proxy, management API), Pydantic 2, httpx, cryptography; Node.js/TypeScript SDK and npm CLI wrapper
Database	SQLite (AES-GCM encrypted pseudonym vault, metadata-only audit store); Supabase PostgreSQL with forced row-level security for the management plane, tested with pgTAP
AI / ML Models & Libraries	scikit-learn context classifier with deterministic rules; regex, Shannon-entropy and fragment-reconstruction detectors; optional Microsoft Presidio Analyzer 2.2.363 with spaCy en_core_web_sm 3.8.0; Gitleaks 8.30.1
Tools & IDE	VS Code, Git and GitHub, GitHub Actions CI (Ubuntu and Windows), pytest, Vitest, Playwright, Docker, Supabase CLI 2.110.0, WSL Ubuntu, OpenCode 1.18.26 (Review 2 demo)
Deployment / Hosting	Local-first: editable pip install, Docker Compose bound to 127.0.0.1:8765, local Supabase stack and private VSIX/npm packages. No hosted deployment yet.

Architecture Diagram

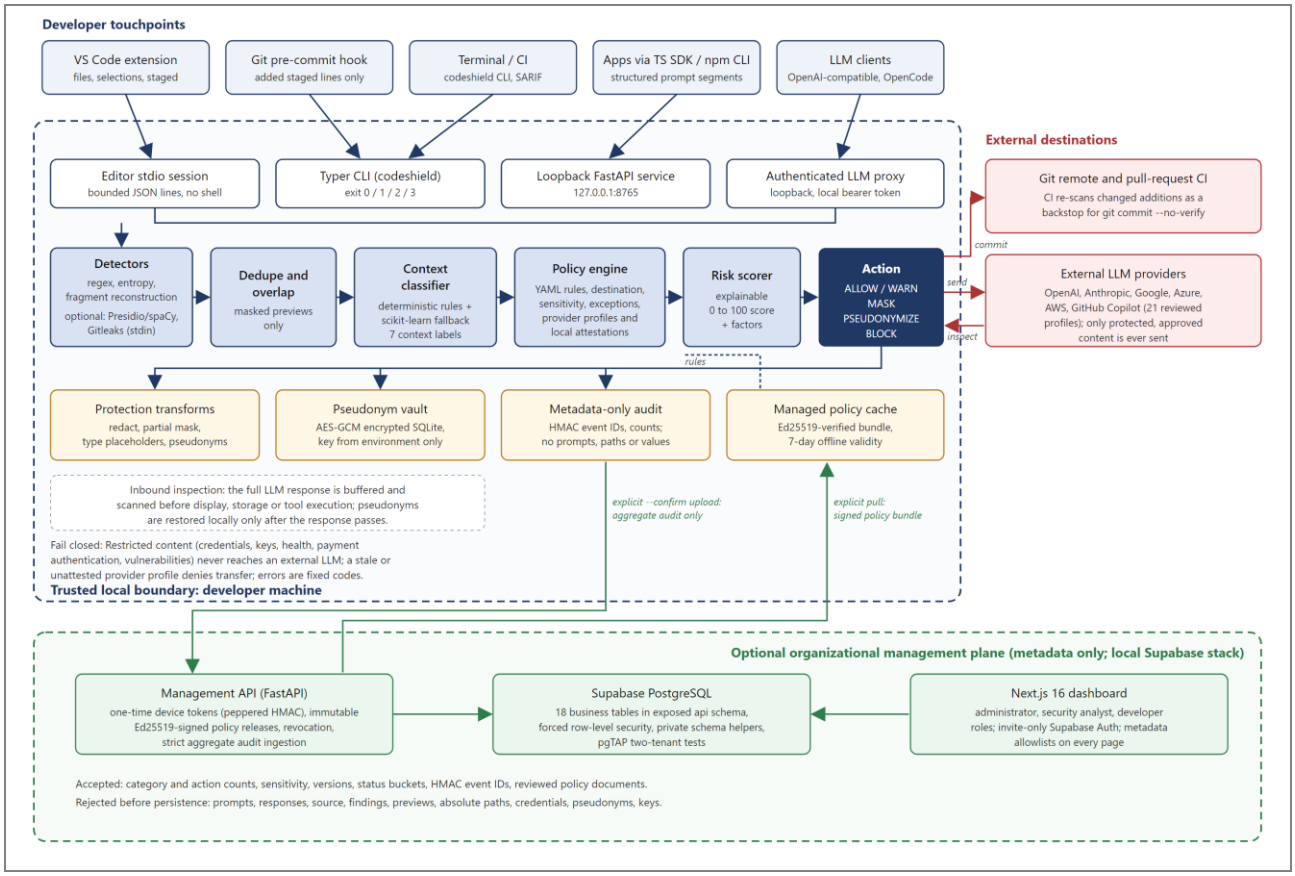


Fig. 1: System architecture of CodeShield AI: local trust boundary, decision pipeline, external destinations and optional management plane

PART B – TEAM AND PROJECT JOURNEY

Particular	Details
Project Type	Single-member

Division of Work and Skills

S. No.	Student Name	Register Number	Work Done	Technical Skills Already Known	Skills, Tools & Knowledge Acquired
1	Bhargav Mahanta	{{Register Number}}	Entire project: requirements, literature survey, architecture, Python engine, Git hook and CI, TypeScript SDK, VS Code extension, LLM proxy, OpenCode demo, management plane and dashboard, evaluation, documentation	{{Skills Already Known}}	Secret and PII detection; scikit-learn evaluation; FastAPI and Typer; AES-GCM, HMAC and Ed25519; Git hooks and SARIF; VS Code extension API; LLM proxy security; Supabase RLS and pgTAP; Next.js; GitHub Actions; TDD

Phases and Plan

Particular	Details
Number of Phases Planned	10 (9 completed, 1 planned)

Phase	Activities	Timeline (From – To)	Status
Phases 1–2	Detectors, risk scoring, CLI and FastAPI; context classifier, YAML policies, masking and encrypted pseudonym vault	Up to 13/07/2026	Completed
Phase 3	Staged Git scanning, managed pre-commit hook, expiring exceptions, npm CLI wrapper, TypeScript SDK, CI	13/07/2026 – 14/07/2026	Completed
Phase 4	Sensitivity model, provider profiles and attestations, LLM preflight, metadata-only audit; Review 1 package	14/07/2026 – 17/07/2026	Completed
Phases 5–6	Presidio/spaCy and Gitleaks adapters, evaluation suites and CI gates; cross-platform builds, SARIF, fragment reconstruction	17/07/2026 – 28/07/2026	Completed
Phase 7	Local VS Code extension with diagnostics, protection and VSIX packaging	28/07/2026 – 29/07/2026	Completed
Phase 8	Authenticated loopback LLM proxy with outbound and inbound inspection	29/07/2026 – 30/07/2026	Completed
Phase 9	Supabase management plane, management API, Next.js dashboard; Review 2 OpenCode demo	30/07/2026 – 07/09/2026	Completed
Stabilization	Line-ending normalization, agent documentation, Review 2 model update, test temp-root fix	08/10/2026	Completed
Phase 10	Re-review stale provider evidence; streaming proxy; deployment and release engineering	After Review 3	Planned

Difficulties Faced in Implementation

S. No.	Difficulty Faced	Where It Went Wrong	How It Was Corrected
1	Date-dependent tests started failing	Exception and attestation fixtures used fixed dates; once the real clock passed them (06/09/2026), four CLI and API tests failed	Fixed date.today() to a set date in the tests only; production expiry logic and fixture dates were not changed
2	Generic credential assignments were missed	The regex rule matched only bare field names, so assignments such as service_api_key = "..." produced no finding	Extended the rule to accept identifier prefixes and added positive and negative regression tests (6 passed)
3	Contract generation differed between runs	Generated OpenAPI and TypeScript contracts were not byte-identical across Windows and Linux, so CI reported drift	Generated contracts in a temporary directory, validated them, then replaced tracked files with rollback; drift checks never write

S. No.	Difficulty Faced	Where It Went Wrong	How It Was Corrected
4	Review 2 model was retired by the provider	OpenAI withdrew gpt-5.3/gpt-5.4 for ChatGPT subscription accounts; the route returned 400 and CodeShield failed closed with CS_OPENCODE_FAILED	Switched the rehearsal to the entitled openai/gpt-5.5 model and kept the rule that no fallback model is ever substituted
5	268 files showed as modified on Windows	Windows tooling on an NTFS mount rewrote LF files as CRLF while the stored blobs stayed LF	Added a repository-wide * text=auto eol=lf rule in .gitattributes and confirmed it with git add --renormalize
6	Runner isolation guard tripped during tests	pytest's temporary root was pinned inside the repository; the OpenCode runner refuses roots below an AGENTS.md file	Removed the basetemp override so the OS temporary directory is used; the guard and its tests were kept unchanged

Review History

Particular	Details
Number of Reviews Attended	2 (Review 1 and Review 2); this report is for Review 3

Review No.	Review Date	Feedback Received	How It Was Addressed
Review 1	05/08/2026	{{Feedback}}	{{Action Taken}}
Review 2	{{DD/MM/YYYY}}	{{Feedback}}	{{Action Taken}}
Review 3	{{DD/MM/YYYY}}	Current review	—

Pitch and Outcomes

Particular	Details
Where Can the Project Be Pitched	Application-security and developer-platform teams adopting AI coding assistants; university innovation and incubation cell or startup demo days; student security conferences and hackathons; the open-source developer-tools community

Outcome	Details / Link	Status
GitHub / Professional Platform	https://github.com/bhargavmahanta/codeshield-ai	In progress
Patent	{{Details}}	{{Status}}
Publication	{{Details}}	{{Status}}
Startup	{{Details}}	{{Status}}
Any Other Plan	{{Details}}	{{Status}}

Status: Planned / In progress / Filed / Published / Not applicable

TABLE OF CONTENTS

ABSTRACT 9
1. INTRODUCTION 9
1.1 Background 9
1.2 Motivation 9
2. PROBLEM STATEMENT 10
3. OBJECTIVES 10
4. LITERATURE SURVEY 10
4.1 Comparison of Related Work 10
4.2 Key Findings 11
4.3 Research Gap 12
5. PROPOSED SYSTEM 12
6. METHODOLOGY 12
6.1 Approach 12
6.2 Dataset Description 13
7. REQUIREMENTS 13
7.1 Functional Requirements 13
7.2 Non-Functional Requirements 14
7.3 Hardware and Software Requirements 14
8. SYSTEM DESIGN 14
8.1 Detailed Design 14
8.2 UML Diagrams 15
9. IMPLEMENTATION 16
9.1 Key Modules 16
9.2 Screenshots 16
10. TESTING AND RESULTS 17
10.1 Test Cases 17
10.2 Performance Metrics 18
10.3 Discussion of Results 19
11. CONCLUSION AND FUTURE WORK 19
11.1 Conclusion 19
11.2 Future Work 20
REFERENCES 20
APPENDIX – SOURCE CODE 21

LIST OF FIGURES

Fig. No.	Caption	Page No.
Fig. 1	System architecture of CodeShield AI: local trust boundary, decision pipeline, external destinations and optional management plane	4
Fig. 2	Use case diagram of CodeShield AI	15
Fig. 3	Sequence diagram of an LLM request through the authenticated local proxy	15
Fig. 4	Managed pre-commit hook blocking a commit that adds a synthetic database credential	16
Fig. 5	Provider-aware LLM preflight blocking a Restricted prompt before OpenCode is started	17
Fig. 6	Local protection replacing sensitive values with type placeholders	17

LIST OF TABLES

Table No.	Caption	Page No.
Table 1	Comparison of related work	10
Table 2	Description of the evaluation datasets	13
Table 3	Hardware and software requirements	14
Table 4	Key modules of the implementation	16
Table 5	Test cases and results	17
Table 6	Performance metrics (Phase 9 suite, CodeShield 0.9.0)	18

ABSTRACT

Developers routinely handle credentials, personal information, proprietary source code and internal configuration while committing to Git and while using large language model (LLM) coding assistants. Conventional secret scanners match fixed patterns or high-entropy strings, but they ignore whether a value is real, where it is going and how the receiving service handles data. CodeShield AI is a local-first framework that combines regular-expression, entropy and fragment-reconstruction detectors (with optional Presidio/spaCy and Gitleaks adapters) with a hybrid rule-based and scikit-learn context classifier, a four-level sensitivity model, destination-aware YAML policies, reviewed LLM-provider profiles and an explainable 0–100 risk score. Each finding results in an allow, warn, mask, pseudonymize or block action before content crosses a Git or LLM trust boundary. Version 0.9.0 provides a Python 3.12 engine and CLI, a loopback FastAPI service, a managed Git pre-commit hook with SARIF output, a TypeScript SDK, a VS Code extension, an authenticated loopback LLM proxy with outbound and inbound inspection, an AES-GCM encrypted pseudonym vault, metadata-only HMAC auditing, and an optional management plane built on Supabase, FastAPI and Next.js that accepts metadata only. On deterministic synthetic suites, the core profile detects every synthetic credential exactly (F2 = 1.00) with no negative-case false positives, the enhanced profile reaches 0.92 exact F1 overall, the context classifier reaches 0.988 macro F1, restricted-transfer and proxy-leak blocking are 100%, and the 100 KB warm p95 latency is 125 ms. The work shows that treating the transfer destination as part of the security decision enables reviewable, local prevention of sensitive-data leakage.

Keywords: Sensitive Data Leakage, Secret Detection, Context-Aware Classification, Destination-Aware Policy, LLM Security, Pseudonymization

1. INTRODUCTION

1.1 Background

Modern software development moves information across several trust boundaries. A value can begin in a local configuration file, become part of a staged Git diff, appear in an application log, or be pasted into an external LLM prompt by a developer or an AI coding assistant. Large-scale studies show that secret leakage is persistent: Meli et al. found secrets exposed across more than 100,000 public GitHub repositories, with thousands of new unique secrets appearing every day [1]. AI coding tools add a second leakage path, because private source and credentials are routinely sent as model context, and code models have been shown to reproduce memorised personal data and credentials [5], [6].

Existing controls are usually separated by channel. Repository secret scanners inspect committed source, data-loss-prevention (DLP) products inspect organisational documents, and prompt filters inspect LLM traffic. Each applies its own rules, and several send the content to a remote service to decide whether it is safe, which discloses the data before the decision is made.

1.2 Motivation

The same string carries a different risk in each destination. A synthetic key in a test fixture may deserve only a warning, while a probable production credential in a public repository or an external prompt must be blocked. A customer's e-mail address may be acceptable after local pseudonymization, whereas a private key must never be rewritten and forwarded. Developers therefore need one explainable policy that understands context and destination, not a collection of unrelated pattern lists.

The rapid adoption of LLM coding assistants also means organisations must reason about provider data handling (retention, training use, zero-data-retention options and stateful features) at the moment a prompt is sent. CodeShield is motivated by the need to make that decision locally, before transmission, while keeping

raw values, keys and reversible mappings on the developer machine and giving security teams only the aggregate metadata they need for governance.

2. PROBLEM STATEMENT

Sensitive data such as credentials, private keys, personal identifiers and proprietary source code leaks from developer workflows through Git commits, logs, API requests, and LLM prompts and responses. Pattern-only scanners raise false positives on placeholders and test data, miss context, and ignore both the destination of the data and the handling characteristics of the receiving provider, while cloud-based classifiers expose the very content they are meant to protect.

This project addresses the problem of designing and implementing a local-first framework that detects sensitive content in files, staged changes, editor buffers and structured LLM traffic; classifies its context and sensitivity; evaluates it against destination-aware and provider-aware policy; and deterministically allows, warns, masks, pseudonymizes or blocks it before it leaves the developer machine. The framework must fail closed whenever policy or provider evidence is missing or stale, and must never log or transmit raw sensitive values.

3. OBJECTIVES

- To detect credentials, personal identifiers, financial values and unknown high-entropy tokens in files, staged Git additions and editor text, reporting only masked previews.
- To distinguish probable real secrets from placeholders, test data, personal data and public information using deterministic context rules and a lightweight scikit-learn classifier, with a target macro F1 of at least 0.90.
- To evaluate every finding against destination, repository visibility, sensitivity, provider profile and local attestation through validated YAML policy and an explainable 0–100 risk score.
- To enforce decisions at the point of action through a managed Git pre-commit hook with a CI backstop, a VS Code extension, and a loopback API and SDK.
- To protect outbound LLM prompts and inspect inbound responses through an authenticated loopback proxy that blocks Restricted content and fails closed on stale or unattested provider evidence.
- To keep raw values, vault keys and pseudonym mappings local, using AES-GCM encrypted reversible pseudonymization and metadata-only HMAC audit records.
- To provide an optional management plane that distributes Ed25519-signed policies and accepts aggregate metadata only, and to measure the whole system with reproducible evaluation gates.

4. LITERATURE SURVEY

4.1 Comparison of Related Work

Table 1: Comparison of related work

S. No.	Paper Title and Year	Methods Used	Advantages	Limitations	Research Gap	How This Project Addresses the Gap
1	How Bad Can It Git? Characterizing Secret Leakage in Public GitHub Repositories (2019) [1]	Pattern-based longitudinal mining of GitHub commits and snapshots	Establishes the scale and causes of secret leakage	Narrow, high-impact key formats; public Git only; detects after publication	No prevention before commit; no context or destination	Scans only added staged lines and blocks before the commit is created, with a CI backstop

S. No.	Paper Title and Year	Methods Used	Advantages	Limitations	Research Gap	How This Project Addresses the Gap
2	Secrets in Source Code: Reducing False Positives Using Machine Learning (2020) [2]	Regex candidates plus contextual features and a voting classifier	Context features reduce alert volume	Decides only whether a value is a secret; no destination or provider model	No policy layer for where the value is going	Hybrid rule and scikit-learn context classifier feeding destination-aware policy and risk
3	SecretBench: A Dataset of Software Secrets (2023) [3]	Curated labelled dataset: 97,479 candidates from 818 repositories	Shared benchmark across 49 languages and 311 file types	Contains real secrets and needs controlled access; no prevention workflow	Safe, shareable evaluation of a prevention tool	Deterministic synthetic suites with versioned manifests, SHA-256 digests and CI gates
4	CoBAn: A Context Based Model for Data Leakage Prevention (2014) [4]	Context term graphs over clusters of confidential documents	Detects embedded or rephrased confidential text	Needs an organisational training corpus and central monitoring	Local DLP without uploading a private corpus	Uses local code context, segment provenance and explicit destination policy only
5	CodexLeaks (2023) [5]; Your Code Secret Belongs to Me (2024) [6]	Extraction and memorisation probes on code LLMs	Show that code models emit personal data and hard-coded credentials	Study model output, not a developer's outbound disclosure	No local transfer gate for prompts and responses	Outbound preflight plus buffered inbound inspection before display or tool use
6	Casper: Prompt Sanitization for Web-Based LLMs (2024) [7]	Browser-side rules, NER and a local model replacing identifiers with placeholders	Client-side privacy without provider changes	Browser and PII focus; no Git, source-code or provider governance	One control for prompts, source and Git	One policy across Git, editor, API and proxy, with reviewed provider profiles
7	Prεεmpt: Sanitizing Sensitive Prompts for LLMs (2025) [8]	Format-preserving encryption and metric differential privacy	Formal treatment of privacy and utility	Transforms tokens without Git or governance controls	When transformation itself is unsafe	Transform-or-block policy: pseudonymize eligible personal data, block Restricted content
8	Benchmarking and Defending Against Indirect Prompt Injection Attacks on LLMs (2023) [9]	BIPIA benchmark across 25 LLMs; boundary-aware defences	Shows that untrusted retrieved content is dangerous	Model-level defences remain imperfect	Enforcement outside the model	Trust-labelled segments; injection indicators with privileged tools are blocked in code

4.2 Key Findings

Five findings shape the design. First, secret leakage is large-scale and persistent, so prevention has to happen before a value enters repository history [1]. Second, regular expressions and entropy are fast candidate generators, but contextual features substantially reduce false positives [2], [4]. Third, code LLMs both receive and emit sensitive data, so both directions of an LLM transfer need inspection [5], [6]. Fourth, local, client-side sanitization is feasible and keeps provider compatibility [7], [8], but transformation is not always safe; a credential replaced by a placeholder can still expose the surrounding code. Fifth, prompt wording cannot act as an authorisation boundary, so least privilege and output validation must be enforced outside the model [9], [12].

Governance frameworks point the same way: the NIST AI Risk Management Framework and the Secure Software Development Framework recommend building these controls into the development life cycle instead of checking at the end [10], [11].

4.3 Research Gap

Existing work addresses individual parts of the problem: repository leakage measurement, false-positive reduction, contextual DLP, model memorisation, prompt sanitization and indirect prompt injection. None of the surveyed systems combines, in a single local developer framework, context classification, destination-sensitive policy, provider data-handling profiles with local attestations, Git and prompt protection, reversible local pseudonymization, inbound response assessment and metadata-only organisational governance. In particular, none treats the transfer destination as a first-class input to the security decision. CodeShield AI addresses this integration gap.

5. PROPOSED SYSTEM

CodeShield AI places a local preflight boundary between developer artefacts and every outbound channel. All entry points (the Typer CLI, the managed Git hook, the VS Code extension's stdio session, the loopback FastAPI service used by the TypeScript SDK, and the authenticated LLM proxy) call the same decision pipeline in the Python engine (Fig. 1).

The pipeline runs regex, Shannon-entropy and static fragment-reconstruction detectors, and optionally Presidio with spaCy for names and locations and Gitleaks over stdin. It resolves overlapping findings and keeps only masked previews. A context classifier labels each finding as a probable real secret, placeholder, test data, personal data, public information, confidential business data or uncertain, using deterministic signals first and a scikit-learn model as a fallback. The policy engine applies validated YAML rules for the destination, repository visibility, sensitivity level and time-bounded exceptions. The risk scorer adds sensitivity, detection confidence, destination exposure, context and policy weight into an explainable 0–100 score (0–29 allow, 30–59 warn, 60–79 mask, 80–100 block), and the stricter of the risk action and the policy action is applied.

For LLM transfers, content is represented as trust-labelled segments (system, developer, user, retrieved, tool output and source code). The provider registry holds 21 reviewed provider-surface profiles, each with an explicit review-due date, and local attestations record an administrative review of the account. Restricted categories are never sent. Eligible personal data is pseudonymized through an AES-GCM encrypted local vault and restored only after the buffered response passes inbound inspection. A missing, stale or unattested profile denies the transfer.

An optional management plane lets an organisation publish immutable, Ed25519-signed policy bundles and collect aggregate audit counts through a separate FastAPI service backed by Supabase PostgreSQL with forced row-level security, viewed through a role-aware Next.js dashboard. The engine pulls policy and uploads audit data only on explicit commands, and the service rejects prompts, source, findings, paths, credentials and pseudonyms before persistence.

6. METHODOLOGY

6.1 Approach

The project followed an incremental, phase-gated method. Each of the nine phases had a written plan and a record of its security decisions (docs/phase-*.md), was built on its own branch, and was merged through a pull request only after the CI gates passed on Ubuntu and Windows with Python 3.12 and Node.js 20 and 22.

Within each phase, behaviour changes were developed test-first: a failing regression test was written before the implementation, as with the credential-assignment regex fix and the OpenCode runner. Security properties such as fail-closed provider checks, fixed error codes, no raw content in any output, and isolated subprocesses with bounded I/O and timeouts were written as tests, not left as conventions.

Detection quality was measured with versioned, deterministic evaluation suites (seed 42). They compute exact-span and relaxed precision, recall, F1 and F2 per category and per detector; false-positive rates on negative cases; context-classifier macro metrics; policy and transformation safety; proxy leak blocking; management rejection recall; and cold and warm latency at 1, 10 and 100 KB. Results are committed as aggregate-only baseline reports, and CI fails when a metric regresses past its threshold.

6.2 Dataset Description

Table 2: Description of the evaluation datasets

Particular	Details
Source	Deterministic synthetic suites generated by the engine (codeshield evaluate prepare): CodeShield synthetic secrets and negatives; person and location sentences following the MIT-licensed Microsoft Presidio Research template method (0.2.5); Git/LLM workflow, fragment, proxy and management cases. No real secrets, real personal data or SecretBench data are used.
Size	Context classifier: 196 training and 84 test records over 7 labels. Phase 9 detection suite: 34 labelled cases (17 core synthetic including 8 negatives, 16 PII entities, 1 adversarial); 20 fragment cases (12 negative); 11 proxy cases (8 negative); 12 management cases; 3 workflow cases; latency inputs of 1, 10 and 100 KB.
Attributes / Features	Input text, entity category (for example API_KEY, JWT, PASSWORD, DATABASE_CONNECTION_STRING, EMAIL, PHONE_NUMBER, PERSON, LOCATION), exact character span, detector source, context label, destination, expected policy action and expected transformation. Classifier features: variable name, path and test-fixture signals, token shape and surrounding words.
Preprocessing	Files are regenerated from a fixed seed and checked against SHA-256 digests in a versioned manifest; generated data is kept outside Git under CODESHIELD_DATA_HOME; features are extracted deterministically; reports keep aggregate metrics only, with no input text, findings, previews or paths.

7. REQUIREMENTS

7.1 Functional Requirements

- Scan files, directories, editor text and only the added lines of staged Git changes, skipping binary and excluded files.
- Detect credentials, private keys, tokens, connection strings, personal and financial identifiers and unknown high-entropy secrets, including secrets split across string fragments.
- Classify context and sensitivity, apply destination-aware YAML policy with expiring path-scoped exceptions, and report an explainable risk score and action with exit codes 0, 1, 2 and 3.
- Produce human, JSON and SARIF 2.1.0 output that contains masked previews only.
- Redact, partially mask, replace with type placeholders or pseudonymize content, and restore pseudonyms locally from the encrypted vault.
- Assess structured LLM prompts against provider profiles and attestations, and proxy OpenAI-compatible Chat Completions and Responses requests with outbound and inbound inspection.
- Install and manage a Git pre-commit hook, and provide VS Code commands for scanning, protection and provider status.
- Publish, verify and activate signed policy bundles, upload aggregate audit metadata only after explicit confirmation, and show it in a role-based dashboard.

7.2 Non-Functional Requirements

- Privacy: raw values, prompts, keys and pseudonym mappings never leave the machine and never appear in logs, errors or audit records; the management plane accepts metadata only.
- Fail-closed security: missing, expired, tampered or unverifiable policy or provider evidence denies the transfer, with no fallback model or authentication route.
- Performance: core scanning of 100 KB within 250 ms warm p95, the enhanced profile within 3 s, and a signed policy pull within 750 ms.
- Accuracy gates: core macro F2 of at least 0.90, core false-positive rate of at most 0.05, context macro F1 of at least 0.90, and 100% of mandatory security outcomes.
- Portability: Windows and Linux, Python 3.12, Node.js 20 and 22, with repeatable builds and generated contracts.
- Isolation: subprocess adapters use a fixed argument list, a minimal environment, stdin-only content, bounded output and a timeout; local services bind to loopback only.
- Explainability and compatibility: every decision reports category, context, policy rule and risk factors; the CLI, API, SDK, SARIF output and policy schema stay backward compatible.

7.3 Hardware and Software Requirements

Table 3: Hardware and software requirements

Category	Requirement
Hardware	64-bit PC or laptop; 8 GB RAM minimum (16 GB recommended for the enhanced spaCy profile and the local Supabase stack); about 2 GB free disk for the core engine and 5 GB with Docker images; no GPU needed; network access only for an approved LLM upstream.
Software	Windows 10/11 or Ubuntu Linux (WSL for the OpenCode demo); Python 3.12; Node.js 20 or 22 with npm; Git; VS Code; Docker Desktop and Supabase CLI 2.110.0 for the management plane; optional Presidio Analyzer 2.2.363, spaCy 3.8 with en_core_web_sm 3.8.0, and Gitleaks 8.30.1; OpenCode 1.18.26 for the Review 2 demo.

8. SYSTEM DESIGN

8.1 Detailed Design

Refer to Fig. 1 in the Project Snapshot. The engine is organised as independent packages under engine/codeshield (detectors, classifier, policies, risk, pseudonymization, storage, git, sarif, llm, providers, proxy, opencode, management and evaluation) that share Pydantic models in models/. Detectors implement a common interface and are composed by a factory according to the selected profile (core or enhanced). The scanner merges their findings with a fixed overlap priority (Gitleaks, regex, Presidio, then entropy). Each finding carries a category, severity, confidence, span and masked preview, never the raw value.

The policy engine loads a versioned YAML document (schema version 2) whose scoring weights and rules are validated at load time. Rule precedence is managed mandatory rules, then managed non-mandatory rules, repository rules and local exceptions, and a local exception can never override a mandatory rule. The LLM service builds structured segment assessments and consults the provider registry; both the proxy and the OpenCode runner call it before any network connection or process is started. Persistence is limited to two local SQLite stores (the encrypted pseudonym vault and the metadata-only audit) and, for the management plane, 18 Supabase tables protected by forced row-level security. Fig. 2 shows the actors and use cases, and Fig. 3 shows the request sequence through the authenticated LLM proxy.

8.2 UML Diagrams

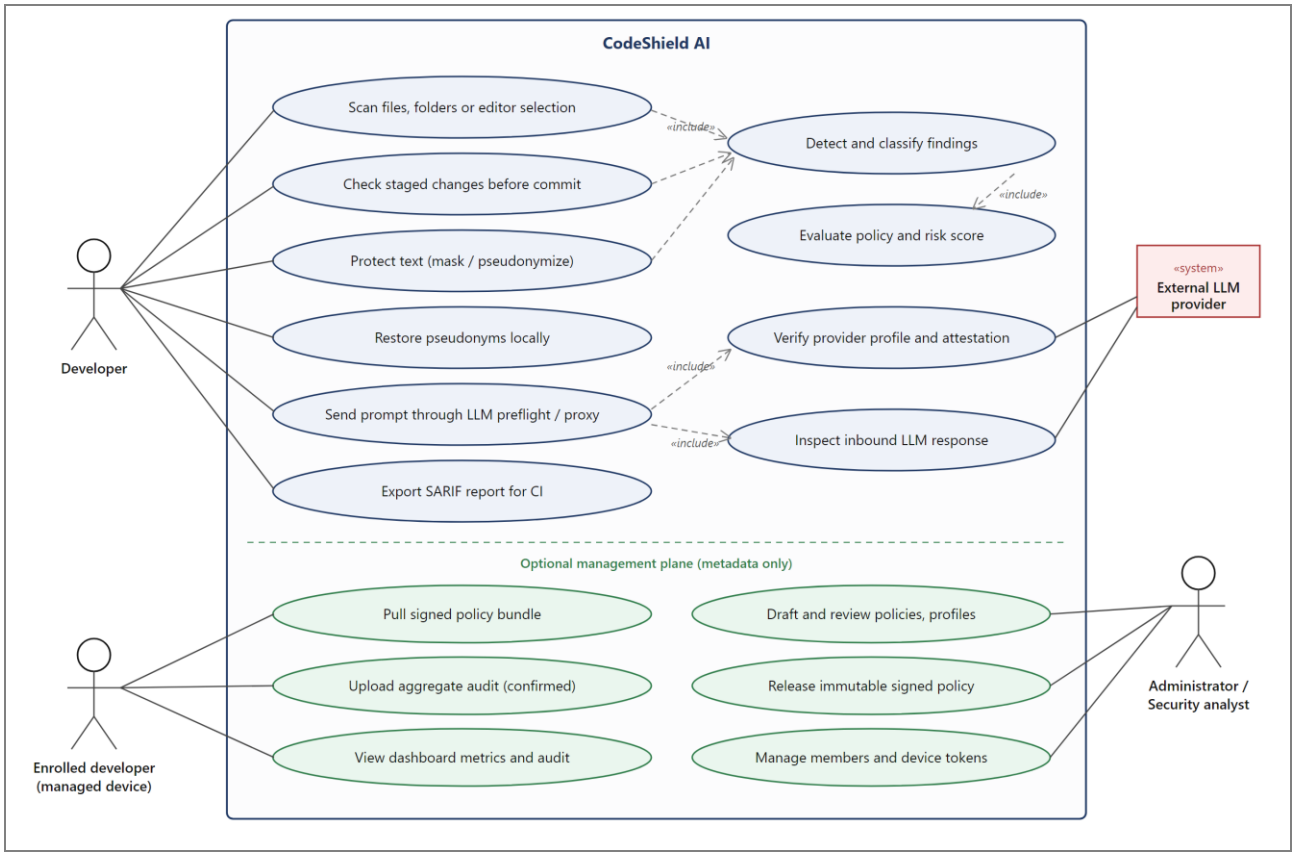


Fig. 2: Use case diagram of CodeShield AI

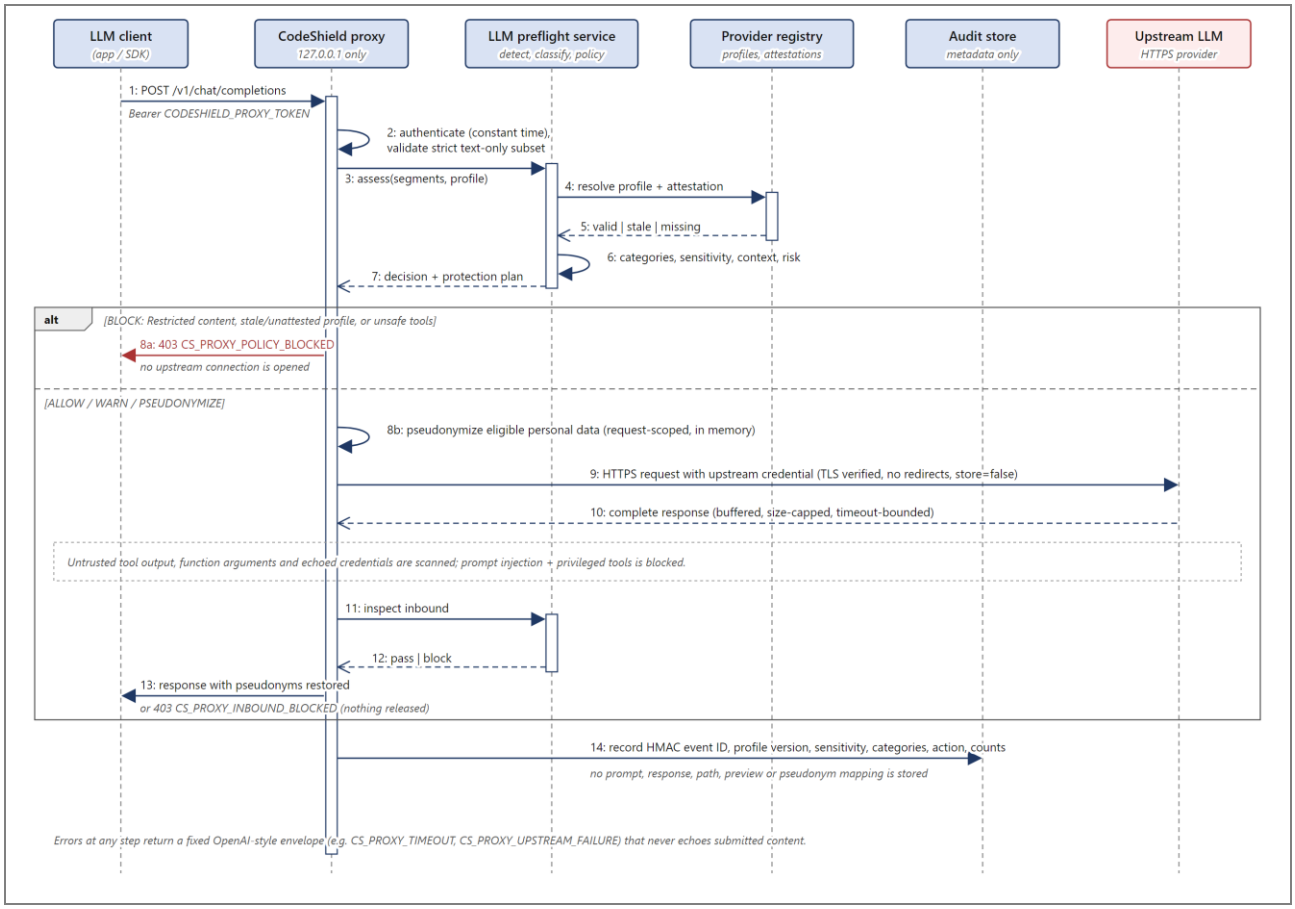


Fig. 3: Sequence diagram of an LLM request through the authenticated local proxy

9. IMPLEMENTATION

9.1 Key Modules

Table 4: Key modules of the implementation

S. No.	Module	Description
1	Detectors (detectors/)	Regex, entropy and fragment detectors; optional lazily loaded Presidio/spaCy and Gitleaks (stdin, private temporary report) adapters; overlap resolution and masking.
2	Context classifier (classifier/)	Deterministic feature extraction and rules with a scikit-learn fallback over 7 context labels; synthetic training data and evaluation.
3	Policy engine and risk scorer (policies/, risk/)	YAML validation, destination rules, expiring exceptions, managed-policy precedence and explainable 0–100 scoring.
4	Protection and vault (pseudonymization/, storage/)	Redaction, partial masking, type placeholders, consistent pseudonyms and the AES-GCM encrypted SQLite vault with local restoration.
5	Git integration and SARIF (git/, sarif/)	Parsing of added staged lines, managed pre-commit hook, CI changed-additions scan and deterministic SARIF 2.1.0.
6	LLM preflight and provider registry (llm/, providers/)	Trust-labelled segment classification, 21 provider profiles, local attestations, inbound assessment and HMAC audit.
7	Authenticated LLM proxy (proxy/)	Loopback OpenAI-compatible Chat Completions and Responses subset with credential replacement, request-scoped pseudonymization, buffered inbound inspection and fixed error codes.
8	OpenCode runner (opencode/)	Review 2 demo route: public-only profile, pinned OpenCode 1.18.26, isolated subprocess with stdin-only content, bounded output and a timeout.
9	Interfaces (cli.py, api/, packages/)	Typer CLI, loopback FastAPI service, generated OpenAPI contracts, TypeScript SDK and npm CLI, and the VS Code extension with diagnostics and protection.
10	Management plane (management/, apps/, supabase/)	Engine client for signed bundle pull and audit upload; FastAPI management API; Supabase schema with RLS and pgTAP tests; Next.js dashboard.

9.2 Screenshots

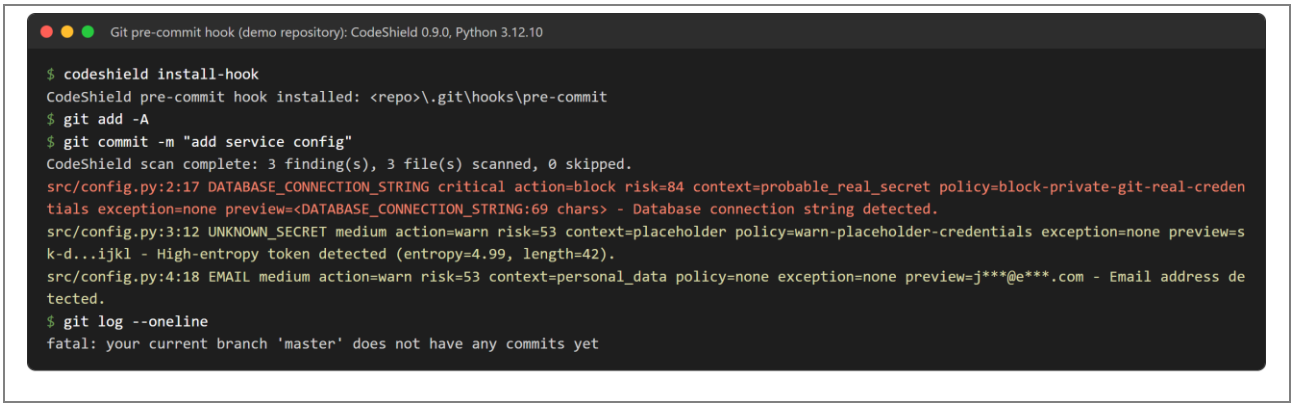


Fig. 4: Managed pre-commit hook blocking a commit that adds a synthetic database credential

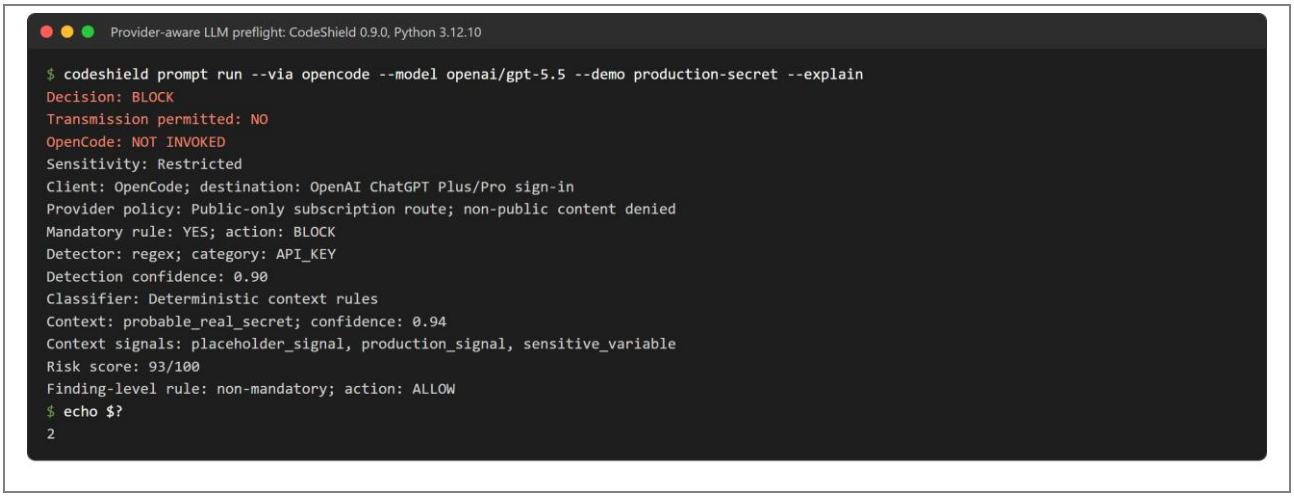


Fig. 5: Provider-aware LLM preflight blocking a Restricted prompt before OpenCode is started

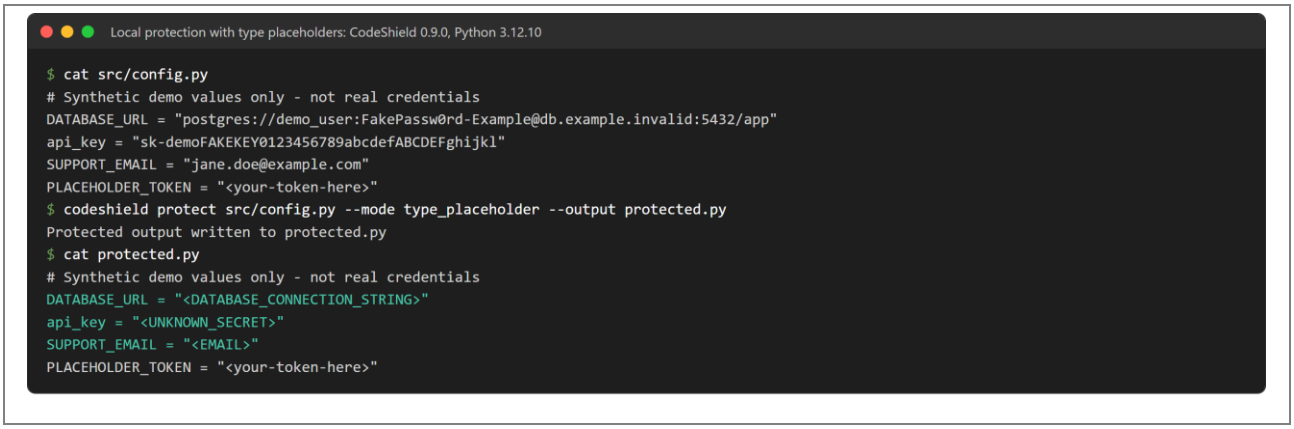


Fig. 6: Local protection replacing sensitive values with type placeholders

10. TESTING AND RESULTS

10.1 Test Cases

Table 5: Test cases and results

Test ID	Description	Input	Expected Output	Actual Output	Status
TC-01	Scan for a public Git destination	Synthetic config.py (DB URL, sk- token, e-mail, placeholder); --destination public_git	DB URL blocked; exit code 2	3 findings; DATABASE_CONNECTION_STRING critical, block, risk 99; token and e-mail masked (risk 68); exit 2	Pass
TC-02	Pre-commit hook on staged credential	Same file staged in a private demo repository; git commit	Commit rejected	Hook reported block (risk 84, rule block-private-git-real-credentials); no commit created	Pass
TC-03	Destination sensitivity	Placeholder-looking token, private Git instead of public Git	Lower action than for public Git	Warn (risk 53, rule warn-placeholder-credentials) instead of mask (risk 68)	Pass
TC-04	Type-placeholder protection	codeshield protect --mode type_placeholder	Sensitive values replaced; placeholder text unchanged	<DATABASE_CONNECTION_STRING>, <UNKNOWN_SECRET>, <EMAIL>; <your-token-here> kept	Pass

Test ID	Description	Input	Expected Output	Actual Output	Status
TC-05	LLM preflight, production-like secret	prompt run --demo production-secret --explain	BLOCK; Restricted; OpenCode not started	BLOCK, Restricted, OpenCode NOT INVOKED, risk 93; exit 2	Pass
TC-06	LLM preflight, test credential	prompt run --demo test-credential --explain	Mandatory rule still blocks	BLOCK, context test_data (0.95), mandatory rule BLOCK; exit 2	Pass
TC-07	Fail closed when OpenCode is unavailable	prompt run --demo safe with no compatible executable	Refuse; no fallback	CS_OPENCODE_INCOMPATIBLE; exit 3	Pass
TC-08	Stale provider evidence	Provider check after review-due date 14/10/2026	Stale profiles reported; transfer denied	20 of 21 profiles stale; check exits 1; proxy returns 403 CS_PROXY_POLICY_BLOCKED	Pass
TC-09	Context classifier accuracy	codeshield evaluate-context (84 test records)	Macro F1 at least 0.90	Macro F1 0.9881; accuracy 0.9881	Pass
TC-10	Engine regression suite	python -m pytest engine/tests (Python 3.12.10, Windows 11)	All tests pass	231 passed, 21 failed, 6 skipped; failures caused by stale provider evidence (TC-08)	Fail

10.2 Performance Metrics

Table 6: Performance metrics (Phase 9 suite, CodeShield 0.9.0)

Metric	Value	Method of Measurement
Core synthetic credential detection, exact F2	1.00 (17 cases)	evaluate run --suite phase9; committed baseline, Windows, Python 3.12.10, seed 42, --profile core
Core negative-case false-positive rate	0.00	evaluate run --suite phase9; committed baseline, Windows, Python 3.12.10, seed 42, --profile core
Overall exact precision / recall / F1, core	0.90 / 0.35 / 0.50	Same run; core does not detect names or locations
Overall exact precision / recall / F1, enhanced	0.92 / 0.92 / 0.92	evaluate run --suite phase9; committed baseline, Windows, Python 3.12.10, seed 42, --profile enhanced
Enhanced PII recall (PERSON, LOCATION)	1.00 (16 of 16)	Enhanced run, Presidio 2.2.363, spaCy 3.8.13
Context classifier macro F1 / accuracy	0.9881 / 0.9881	codeshield evaluate-context; reproduced 22/10/2026
Policy action accuracy / restricted-transfer blocking	1.00 / 1.00	Workflow suite
Fragment exact recall / false-positive rate	1.00 / 0.00	Fragment suite (20 cases)
Proxy leak recall / block recall / false-positive rate	1.00 / 1.00 / 0.00	Proxy suite, local mock upstream (11 cases)
Management rejection recall (signature, tamper, expiry, rollback / prohibited ingestion)	1.00 / 1.00	Management suite (12 cases)
100 KB warm p95 latency	125.49 ms core (budget 250); 830.33 ms enhanced (budget 3,000)	Latency suite, 1/10/100 KB inputs

Metric	Value	Method of Measurement
Signed policy pull, verify and activate (137,018 bytes), warm p95	60.9 ms (budget 750 ms)	npm run benchmark:management, 30/07/2026

10.3 Discussion of Results

The results show the layered design behaving as intended on controlled inputs. The core profile detects every synthetic credential category exactly and raises no false positives on negative cases. Its overall recall (0.35) is low because names and locations are outside its scope; enabling the enhanced Presidio/spaCy profile raises overall exact F1 from 0.50 to 0.92, at the cost of about 0.8 s warm latency per 100 KB, which is why core remains the default. The context classifier separates placeholder, test and real-secret contexts with 0.988 macro F1, misclassifying one uncertain sample as personal data.

The demonstrations in Figs. 4 to 6 confirm the destination-sensitive behaviour. The same placeholder-shaped token was masked for a public Git destination (risk 68) but only warned for a private repository (risk 53), while the database credential was blocked in both cases. In the LLM preflight, even a token correctly recognised as test data was blocked, because the mandatory Restricted rule cannot be weakened by context. With no compatible OpenCode executable, the safe preset failed closed with CS_OPENCODE_INCOMPATIBLE instead of falling back.

Re-running the engine test suite on 22/10/2026 gave 231 passed, 21 failed and 6 skipped (the skips are the uninstalled Presidio evaluator and Linux-only process tests). All 21 failures are provider-dependent (19 proxy tests, one API metadata test and one provider CLI test) and coincide with the provider-evidence review-due date of 14/10/2026: 20 of the 21 provider profiles are now reported as stale, so the proxy refuses transfers (HTTP 403 CS_PROXY_POLICY_BLOCKED) where the tests expect a reviewed profile. This is the fail-closed design working as specified, but the evidence has to be re-reviewed against current provider documentation before the next release; simply extending the dates would defeat the control. The Review 2 record reported 257 passed and 6 skipped before the expiry.

All accuracy figures come from deterministic synthetic data and do not estimate real-world precision or recall. The split-token adversarial case is still missed under exact matching, although relaxed overlap matching detects it.

11. CONCLUSION AND FUTURE WORK

11.1 Conclusion

CodeShield AI shows that sensitive-data leakage in developer and LLM workflows can be treated as a context and destination problem and prevented locally. Version 0.9.0 integrates detection, context classification, sensitivity, destination-aware policy, provider evidence, explainable risk and protective actions behind a single local boundary that is reachable from the terminal, Git, the editor, applications and LLM clients.

On the synthetic suites the system detects every synthetic credential with no negative false positives, reaches 0.92 F1 with the enhanced detectors and 0.988 macro F1 for context classification, and blocks all restricted transfers and proxy leaks within its latency budgets. The optional management plane gives organisations signed policy distribution and aggregate oversight without receiving any content. The main contribution is this integration and the explicit use of the transfer destination in the security decision; the fail-closed behaviour was itself observed when the provider evidence expired.

11.2 Future Work

Planned work after Review 3:

(1) Re-review the 20 stale provider profiles against current first-party documentation and record new evidence before release.

(2) Evaluate on licensed real-world corpora such as SecretBench under controlled access to estimate real precision and recall.

(3) Add streaming support to the proxy with incremental inbound inspection.

(4) Host the management plane with an enterprise identity provider, backup and restore, and release engineering, after a separate security review.

(5) Add multilingual PII models and stronger detection of split and obfuscated secrets.

(6) Add a logging sanitizer, and publish the VS Code extension and npm packages after review.

REFERENCES

[1] M. Meli, M. R. McNiece, and B. Reaves, "How Bad Can It Git? Characterizing Secret Leakage in Public GitHub Repositories," in Proc. Network and Distributed System Security Symposium (NDSS), 2019, doi: 10.14722/ndss.2019.23418.

[2] A. Saha, T. Denning, V. Srikumar, and S. K. Kasera, "Secrets in Source Code: Reducing False Positives Using Machine Learning," in Proc. 12th Int. Conf. Communication Systems & Networks (COMSNETS), 2020, pp. 168–177.

[3] S. K. Basak, L. Neil, B. Reaves, and L. Williams, "SecretBench: A Dataset of Software Secrets," in Proc. 20th Int. Conf. Mining Software Repositories (MSR), Data and Showcase Track, 2023.

[4] G. Katz, Y. Elovici, and B. Shapira, "CoBAn: A Context Based Model for Data Leakage Prevention," Information Sciences, vol. 262, pp. 137–158, 2014, doi: 10.1016/j.ins.2013.10.005.

[5] L. Niu, S. Mirza, Z. Maradni, and C. Pöpper, "CodexLeaks: Privacy Leaks from Code Generation Language Models in GitHub Copilot," in Proc. 32nd USENIX Security Symposium, 2023, pp. 2133–2150.

[6] Y. Huang, Y. Li, W. Wu, J. Zhang, and M. R. Lyu, "Your Code Secret Belongs to Me: Neural Code Completion Tools Can Memorize Hard-Coded Credentials," Proc. ACM on Software Engineering, 2024, doi: 10.1145/3660818.

[7] C. J. Chong, C. Hou, Z. Yao, and S. M. S. Talebi, "Casper: Prompt Sanitization for Protecting User Privacy in Web-Based Large Language Models," arXiv:2408.07004, 2024.

[8] A. R. Chowdhury, D. Glukhov, D. Anshumaan, P. Chalasani, N. Papernot, S. Jha, and M. Bellare, "Prεεmpt: Sanitizing Sensitive Prompts for LLMs," arXiv:2504.05147, 2025.

[9] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, "Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models," arXiv:2312.14197, 2023.

[10] National Institute of Standards and Technology, Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1, Jan. 2023, doi: 10.6028/NIST.AI.100-1.

[11] National Institute of Standards and Technology, Secure Software Development Framework (SSDF) Version 1.1, NIST SP 800-218, Feb. 2022, doi: 10.6028/NIST.SP.800-218.

[12] OWASP Foundation, "LLM01: Prompt Injection," OWASP Top 10 for Large Language Model Applications, 2025. [Online]. Available: https://genai.owasp.org/llmrisk/llm01-prompt-injection/

APPENDIX – SOURCE CODE

```
Repository: https://github.com/bhargavmahanta/codeshield-ai (main, commit 297b9e1, version 0.9.0)

engine/codeshield/   Python engine: detectors, classifier, policies, risk, pseudonymization,
                     storage, git, sarif, llm, providers, proxy, opencode, management, api, cli.py
engine/tests/        pytest suites (detectors, policy, CLI, API, proxy, OpenCode, management)
packages/            shared contracts, npm CLI wrapper, TypeScript SDK, VS Code extension
apps/                management-api (FastAPI) and dashboard (Next.js 16)
supabase/            migration, seed metadata and pgTAP tests
policies/, reports/  default policy and committed evaluation baselines

Excerpt: engine/codeshield/risk/scoring.py (explainable risk score)

        score = min(
            100,
            sensitivity
            + confidence
            + exposure
            + context_contribution
            + repository
            + policy_importance,
        )
        severity, risk_action = severity_and_action(score)
        action = (
            RecommendedAction.WARN
            if decision.exception_id is not None
            else _stricter_action(risk_action, decision.action)
        )

def severity_and_action(score: int) -> tuple[Severity, RecommendedAction]:
    if score <= 29:
        return Severity.LOW, RecommendedAction.ALLOW
    if score <= 59:
        return Severity.MEDIUM, RecommendedAction.WARN
    if score <= 79:
        return Severity.HIGH, RecommendedAction.MASK
    return Severity.CRITICAL, RecommendedAction.BLOCK
```